UNIVERSIDAD TECNOLÓGICA DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA CARRERA DE INGENIERÍA DE SOFTWARE

PRUEBAS DE SOFTWARE (100000S13F)

AVANCE PARCIAL 1 (APF1)

Sistema Evaluado: CamShop — Plataforma de Comercio Electrónico y Monitoreo Inteligente

Empresa Titular: CamShop Seguridad & Tecnología S.A.C. (RUC: 20609823411)

Autores:	• Lujan Aderiano, Arnold Steven (U18207053) • Gonzales Pocco, Jesus Martin (U24308236)
Docente:	**Iquise Peralta, Juan Manuel**
Carrera / Ciclo:	Ingeniería de Software · Ciclo 9
Lugar y Fecha:	Lima, Perú · Octubre – 2026

TABLA DE CONTENIDO

CAPÍTULO 1: INTRODUCCIÓN, CONTEXTO Y DOCUMENTOS DE ENTRADA 3
1.1 Descripción de la Empresa y Dominio de Negocio 3
1.2 Problemática del Negocio y Justificación del Software 3
1.3 Documentos de Entrada (Base del Proyecto de Pruebas) 3
1.4 Especificación de Requisitos de Software e Historias de Usuario 4
1.4.1 Requisitos Funcionales del Sistema 4
1.4.2 Requisitos No Funcionales del Sistema 5
1.4.3 Historias de Usuario y Reglas de Negocio 6
1.5 Arquitectura de Software y Diagramas del Sistema 8
1.5.1 Arquitectura Desacoplada y Stack Tecnológico 8
1.5.2 Diagrama de Arquitectura y Componentes del Sistema 8
1.5.3 Diagrama del Modelo de Datos Relacional 9
1.5.4 Diagrama de Flujo de Servicios Integrados 10
1.6 Objetivos del Proyecto de Aseguramiento de la Calidad 12
CAPÍTULO 2: FASE 1 — PLAN GENERAL DE PRUEBAS (PLANIFICACIÓN) 13
2.1 Documento de Alcance de Pruebas: In-Scope y Out-of-Scope 13
2.2 Tipos de Prueba a Aplicar y Justificación Técnica 13
2.3 Entorno de Pruebas: Hardware, Software, Datos y Herramientas 14
2.4 Cronograma General del Proyecto 15
2.5 Matriz de Gestión de Riesgos del Producto y Riesgo Operacional por Módulo 16
2.5.1 Matriz de Riesgos Técnicos del Proyecto 16
2.5.2 Matriz de Riesgos Operacionales y Cobertura de Pruebas por Módulo 16
2.6 Criterios de Entrada, Salida y Suspensión / Reanudación 17
CAPÍTULO 3: FASE 2 — DISEÑO DE CASOS DE PRUEBA (UNIDAD 1) 18
3.1 Matriz de Trazabilidad y Técnicas de Diseño Aplicadas 18
3.2 Catálogo de Casos de Prueba Unitarios Backend 18
3.3 Catálogo de Casos de Prueba Unitarios Frontend (Jasmine + Karma) 21
3.4 Catálogo de Casos de Prueba de Integración Backend 23
3.5 Catálogo de Casos de Prueba de Integración Frontend 25
CAPÍTULO 4: FASE 2 — DISEÑO DE CASOS DE PRUEBA (UNIDAD 2) 26
4.1 Catálogo de Casos de Prueba Funcionales E2E 26
REFERENCIAS BIBLIOGRÁFICAS 27

CAPÍTULO 1: INTRODUCCIÓN, CONTEXTO Y DOCUMENTOS DE ENTRADA

1.1 Descripción de la Empresa y Dominio de Negocio

CamShop Seguridad & Tecnología S.A.C. (RUC: 20609823411) es una empresa peruana especializada en la provisión e integración de soluciones tecnológicas de videovigilancia inteligente, cámaras IP de alta definición con algoritmos de visión artificial, sistemas domo antivandálicos para exteriores, cerraduras domóticas y sistemas motorizados PTZ de 360 grados con rastreo autónomo.

El mercado objetivo abarca tanto el sector residencial (familias que buscan proteger sus hogares mediante monitoreo remoto en tiempo real) como el sector corporativo y Mypes (empresas comerciales, almacenes y oficinas que requieren auditoría visual continua y evidencia digital con alta fidelidad).

1.2 Problemática del Negocio y Justificación del Software

Durante los últimos años, CamShop operó bajo un esquema manual y descentralizado basado en mostradores físicos y atención por mensajería instantánea (WhatsApp). Dicho esquema generó fallas operativas severas:

- • **Quiebre de Stock:** Desincronización permanente entre el inventario físico de almacén y las ventas confirmadas, provocando ventas sin stock y quiebres de existencias.
- • **Errores en Cotizaciones:** Errores recurrentes de cálculo en cotizaciones con descuentos y subtotales por falta de un motor centralizado de validación aritmética.
- • **Pérdida de Trazabilidad:** Inexistencia de trazabilidad cronológica del estado de los pedidos, imposibilitando notificar al cliente el estado real de su compra.
- • **Riesgo de Seguridad:** Vulnerabilidad en el control de acceso a datos sensibles de clientes y cotizaciones comerciales.

Para erradicar estos cuellos de botella, se proyectó el desarrollo de la plataforma web CamShop, un sistema integral de comercio electrónico y monitoreo que digitaliza todo el ciclo de vida del cliente: catálogo reactivo, carrito de compras sincronizado, cálculo estricto de subtotales, checkout transaccional con pago simulado y panel administrativo de control de pedidos e inventario.

1.3 Documentos de Entrada

Conforme al estándar internacional ISTQB (International Software Testing Qualifications Board) y la norma IEEE 829, el proceso de pruebas requiere de una Base de Pruebas (Test Basis) sólida y formalizada que actúe como fuente de verdad para la derivación de casos de prueba, condiciones de evaluación y criterios de aceptación.

Los Documentos de Entrada que gobiernan y sustentan el esfuerzo de aseguramiento de calidad de CamShop son los siguientes:

- **1. Especificación de Requisitos de Software e Historias de Usuario:** Especificación formal que define los requerimientos funcionales y no funcionales del sistema, complementada por Historias de Usuario con escenarios de prueba Given-When-Then y reglas de negocio operacionales.

- **2. Documento de Alcance de Pruebas:** Documento que establece con precisión las fronteras operativas del sistema, definiendo qué módulos y flujos están incluidos (In-Scope) y cuáles quedan excluidos (Out-of-Scope) del ciclo de pruebas.
- **3. Arquitectura de Software y Diagramas del Sistema:** Diseño formal de la solución desacoplada cliente-servidor, diagramas de componentes de integración y el modelo de base de datos relacional, indispensables para diseñar pruebas unitarias y de integración.
- **4. Cronograma General del Proyecto:** Planificación temporal que articula y alinea los sprints de desarrollo, las entregas de versiones (releases), las ventanas de pruebas continuas y los despliegues automatizados en CI/CD.
- **5. Matriz de Gestión de Riesgos:** Evaluación analítica del impacto y probabilidad de fallo por módulo de software, permitiendo enfocar los recursos de QA en las zonas de mayor riesgo crítico (Risk-Based Testing).

1.4 Especificación de Requisitos de Software e Historias de Usuario

A continuación se formaliza el conjunto de requisitos funcionales y no funcionales que rigen el comportamiento esperado de la plataforma:

1.4.1 Requisitos Funcionales del Sistema

Código	Nombre del Requisito	Descripción Funcional y Operativa	Prioridad
RF-01	**Registro de Usuarios**	El sistema debe permitir a nuevos clientes registrarse con correo electrónico único y contraseña segura (mínimo 6 caracteres). Encripta con BCrypt y asigna rol ROLE_CLIENTE.	**Alta**
RF-02	**Autenticación y Sesión JWT**	El sistema debe validar credenciales de acceso, rechazar intentos fraudulentos y generar tokens JWT firmados digitalmente con expiración controlada.	**Alta**
RF-03	**Catálogo y Búsqueda Reactiva**	El sistema debe desplegar el catálogo de cámaras con filtrado en tiempo real por categoría (Domo, Bala, PTZ), rango de precio y disponibilidad de existencias.	**Alta**
RF-04	**Gestión de Carrito de Compras**	El sistema debe permitir agregar, modificar cantidades y remover cámaras del carrito, verificando que la cantidad demandada no exceda el stock físico real.	**Alta**
RF-05	**Checkout y Generación de Órdenes**	El sistema debe convertir el carrito activo en un pedido formal, aplicar transacción ACID, descontar unidades del stock de almacén y vaciar el carrito.	**Alta**
RF-06	**Pasarela de Pago Simulado**	El sistema debe simular el procesamiento de cobro electrónico, validar formato de tarjeta de prueba y actualizar el estado de la orden a PAGADA.	**Media**
RF-07	**Reseñas y Calificaciones**	El sistema debe permitir a usuarios registrados ingresar puntuaciones (1 a 5 estrellas) y comentarios, recalculando el promedio público de cada cámara.	**Media**
RF-08	**Panel Administrativo de Control**	El sistema debe restringir el acceso exclusivo al rol ADMIN para crear cámaras, modificar precios, actualizar stock y cambiar el estado logístico de órdenes.	**Media**

1.4.2 Requisitos No Funcionales del Sistema

Código	Atributo de Calidad	Especificación y Criterio de Aceptación Cuantitativo
RNF-01	**Seguridad Criptográfica**	Cifrado de contraseñas con función hash BCrypt (factor de trabajo >= 10). Transmisión de credenciales y autorización vía encabezados HTTP con tokens firmados HMAC-SHA256.
RNF-02	**Rendimiento y Tiempo de Respuesta**	Las consultas GET de catálogo y operaciones de carrito deben responder en un percentil 95 por debajo de 1.2 segundos bajo carga estándar.
RNF-03	**Consistencia Transaccional (ACID)**	Toda operación de generación de pedidos debe ejecutarse bajo contexto transaccional (@Transactional); ante cualquier excepción se revierte el stock mediante rollback.
RNF-04	**Diseño Responsivo y Usabilidad**	La interfaz construida con Angular Signals y Tailwind CSS debe renderizar fluidamente en pantallas desde 360px (smartphones) hasta 4K UHD sin desbordamientos visuales.
RNF-05	**Mantenibilidad y Cobertura de Pruebas**	El código debe mantener separación estricta por capas y alcanzar un umbral mínimo de cobertura de código ejecutable de 80% en los módulos de negocio centrales.

1.4.3 Historias de Usuario y Reglas de Negocio

Las Historias de Usuario estructuran los casos de uso desde la perspectiva del usuario final y del negocio, definiendo con exactitud las reglas operativas y los criterios de aceptación (formato Given-When-Then) requeridos para el diseño de pruebas:

Historia de Usuario:	**HU-01: Registro y Autenticación Segura de Clientes**
Descripción:	Como cliente particular o representante de empresa, quiero registrarme e iniciar sesión con credenciales seguras, para gestionar mis compras y proteger mi historial de pedidos.
Reglas de Negocio (RN):	• RN-01.1: El correo electrónico debe poseer formato válido estándar (RFC 5322) y ser único en el sistema. • RN-01.2: La contraseña debe tener al menos 6 caracteres y ser hasheada irreversiblemente con BCrypt. • RN-01.3: El login exitoso genera un token JWT con tiempo de expiración de 24 horas y asigna rol ROLE_CLIENTE.
Criterios de Aceptación: (Given-When-Then)	Escenario 1 (Registro Exitoso): Dado que un usuario ingresa un correo no registrado y contraseña >= 6 caracteres, cuando presiona "Registrarse", entonces el sistema guarda el usuario con clave cifrada y retorna código HTTP 200/201. Escenario 2 (Rechazo por Correo Duplicado): Dado que el correo ya existe en base de datos, cuando el usuario intenta registrarse nuevamente, entonces el sistema deniega el registro, lanza ConflictException y no persiste datos.

Historia de Usuario:	**HU-02: Exploración de Catálogo y Filtrado Dinámico**
Descripción:	Como usuario interesado en seguridad electrónica, quiero filtrar el catálogo por categoría y rango de precios, para encontrar rápidamente cámaras acordes a mi presupuesto y necesidad técnica.
Reglas de Negocio (RN):	• RN-02.1: El catálogo únicamente muestra cámaras activas y en stock salvo que se aplique filtro especial. • RN-02.2: El rango de precio mínimo no puede ser mayor que el precio máximo (precioMin <= precioMax). • RN-02.3: La reactividad en frontend se maneja mediante Angular Signals para evitar re-renderizados pesados.
Criterios de Aceptación: (Given-When-Then)	Escenario 1 (Filtrado por Rango de Precios): Dado que existen productos con precios S/. 100, S/. 250 y S/. 600, cuando el usuario filtra con precioMin=150 y precioMax=400, entonces el catálogo muestra exclusivamente los productos en dicho intervalo (cámara de S/. 250).

Historia de Usuario:	**HU-03: Control de Existencias y Carrito Reactivo**

Descripción:	Como comprador potencial, quiero agregar cámaras al carrito y ajustar cantidades, para calcular el costo total de mi compra sin exceder el inventario disponible.
Reglas de Negocio (RN):	• RN-03.1: No se permite agregar una cantidad mayor al stock físico existente en almacén. • RN-03.2: El subtotal de cada ítem es el resultado exacto de: (Precio Unitario * Cantidad). • RN-03.3: Al cambiar la cantidad, el total acumulado debe recalcularse de inmediato.
Criterios de Aceptación: (Given-When-Then)	Escenario 1 (Validación de Stock Límite): Dado que una cámara cuenta con 5 unidades en stock, cuando el usuario solicita comprar 6 unidades, entonces el sistema bloquea la acción, arrojando IllegalArgumentException por falta de stock. Escenario 2 (Cálculo Correcto de Subtotal): Dado un producto con precio S/. 200.00, cuando se agregan 3 unidades, entonces el subtotal del ítem es exactamente S/. 600.00.

Historia de Usuario:	**HU-04: Transacción de Compra y Descuento Atómico de Stock**
Descripción:	Como cliente con carrito activo, quiero procesar la orden de compra y simular el pago, para confirmar mi pedido y asegurar la reserva de los equipos adquiridos.
Reglas de Negocio (RN):	• RN-04.1: La creación de la orden debe ser atómica y transaccional (@Transactional). • RN-04.2: Se debe decrementar el stock de cada producto involucrado en la cantidad exacta comprada. • RN-04.3: Tras completar el pedido, el carrito del usuario debe quedar en estado vacío (0 ítems). • RN-04.4: Si la simulación de cobro o la persistencia fallan, el inventario vuelve a su estado original (rollback).
Criterios de Aceptación: (Given-When-Then)	Escenario 1 (Creación Exitosa de Orden con Descuento de Stock): Dado un producto con 10 unidades y un carrito con 2 unidades, cuando el usuario confirma la orden con token JWT válido, entonces la orden se registra con estado PAGADA, el carrito se vacía y el stock del producto pasa a 8 unidades.

1.5 Arquitectura de Software y Diagramas del Sistema

Para planificar y diseñar casos de prueba con exactitud (especialmente las pruebas de integración entre capas y servicios REST), es indispensable formalizar la arquitectura de componentes, las rutas de comunicación y el modelo de datos relacional de la plataforma.

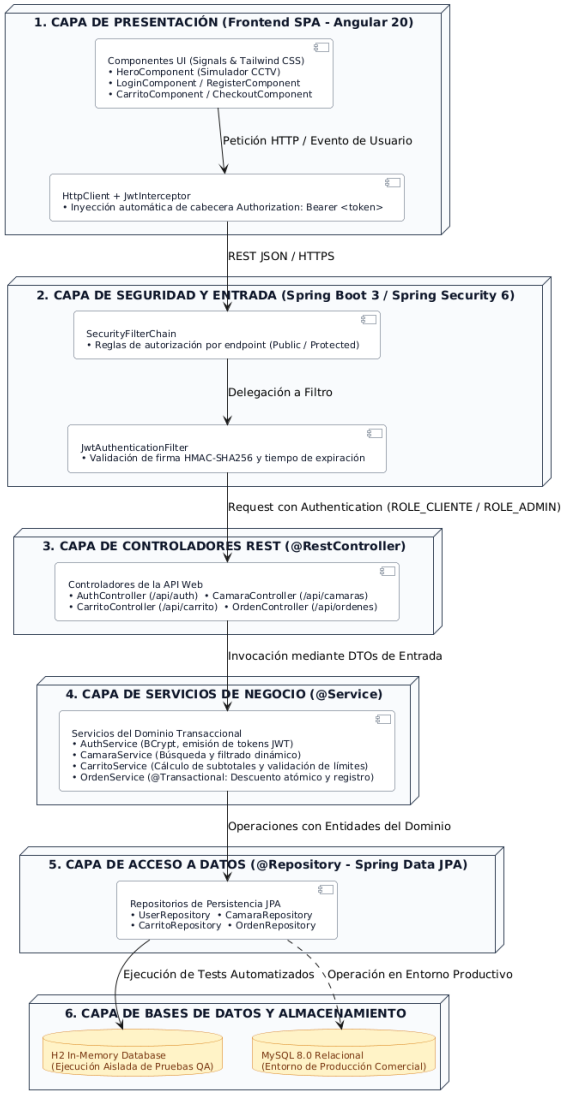
1.5.1 Arquitectura Desacoplada y Stack Tecnológico

CamShop implementa el patrón arquitectural de capas desacopladas (Clean Architecture / 3-Tier Web) comunicadas mediante protocolo HTTP y formato de intercambio JSON:

- • **Capa de Presentación (Frontend):** Single Page Application basada en componentes desacoplados (Angular 20 Standalone), gestión reactiva de estado mediante Angular Signals, interceptores de seguridad HTTP (JwtInterceptor) y hojas de estilo utilitarias con Tailwind CSS.
- • **Capa de Negocio y Seguridad (Backend):** Servicio web RESTful construido sobre Java 17/21 y Spring Boot 3. El pipeline de seguridad se implementa con Spring Security 6, que intercepta peticiones mediante un filtro JwtAuthenticationFilter antes de alcanzar los controladores REST.
- • **Capa de Persistencia y Datos:** Gestión de entidades mediante Spring Data JPA e Hibernate. En producción se utiliza MySQL 8.0 y en entornos de pruebas automatizadas se utiliza H2 Database en memoria, garantizando aislamiento total entre ejecuciones.

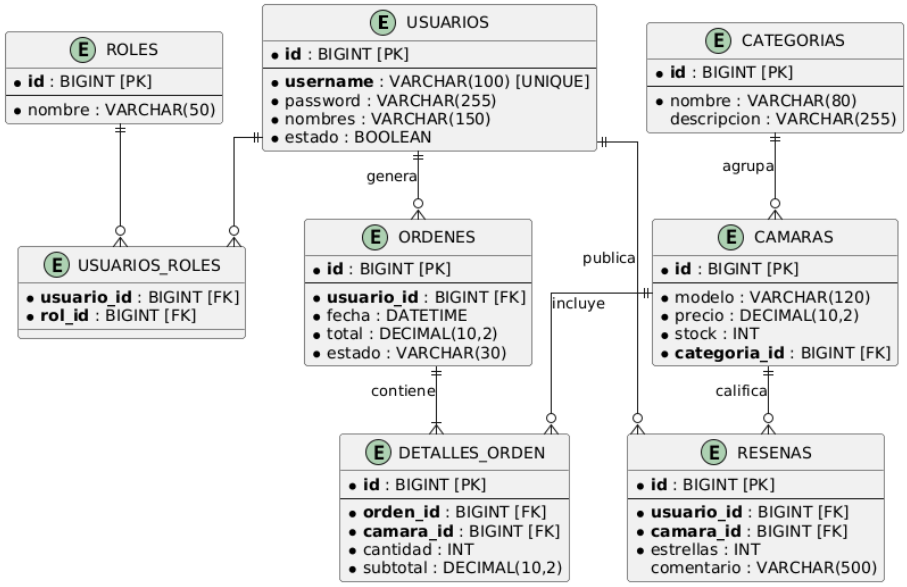
1.5.2 Diagrama de Arquitectura y Componentes del Sistema

El siguiente diagrama estructurado ilustra la topología de componentes y el flujo de comunicación que orienta el diseño de casos de prueba unitarios y de integración:


1. CAPA DE PRESENTACIÓN (Frontend SPA - Angular 20)
Componentes UI (Signals & Tailwind CSS)
• HeroComponent (Simulador CCTV)
• LoginComponent / RegisterComponent
• CarritoComponent / CheckoutComponent
Petición HTTP / Evento de Usuario
HttpClient + JwtInterceptor
• Inyección automática de cabecera Authorization: Bearer <token>
REST JSON / HTTPS
2. CAPA DE SEGURIDAD Y ENTRADA (Spring Boot 3 / Spring Security 6)
SecurityFilterChain
• Reglas de autorización por endpoint (Public / Protected)
Delegación a Filtro
JwtAuthenticationFilter
• Validación de firma HMAC-SHA256 y tiempo de expiración
Request con Authentication (ROLE_CLIENTE / ROLE_ADMIN)
3. CAPA DE CONTROLADORES REST (@RestController)
Controladores de la API Web
• AuthController (/api/auth) • CamaraController (/api/camaras)
• CarritoController (/api/carrito) • OrdenController (/api/ordenes)
Invocación mediante DTOs de Entrada
4. CAPA DE SERVICIOS DE NEGOCIO (@Service)
Servicios del Dominio Transaccional
• AuthService (BCrypt, emisión de tokens JWT)
• CamaraService (Búsqueda y filtrado dinámico)
• CarritoService (Cálculo de subtotales y validación de límites)
• OrdenService (@Transactional: Descuento atómico y registro)
Operaciones con Entidades del Dominio
5. CAPA DE ACCESO A DATOS (@Repository - Spring Data JPA)
Repositorios de Persistencia JPA
• UserRepository • CamaraRepository
• CarritoRepository • OrdenRepository
Ejecución de Tests Automatizados
Operación en Entorno Productivo
6. CAPA DE BASES DE DATOS Y ALMACENAMIENTO
H2 In-Memory Database
(Ejecución Aislada de Pruebas QA)
MySQL 8.0 Relacional
(Entorno de Producción Comercial)

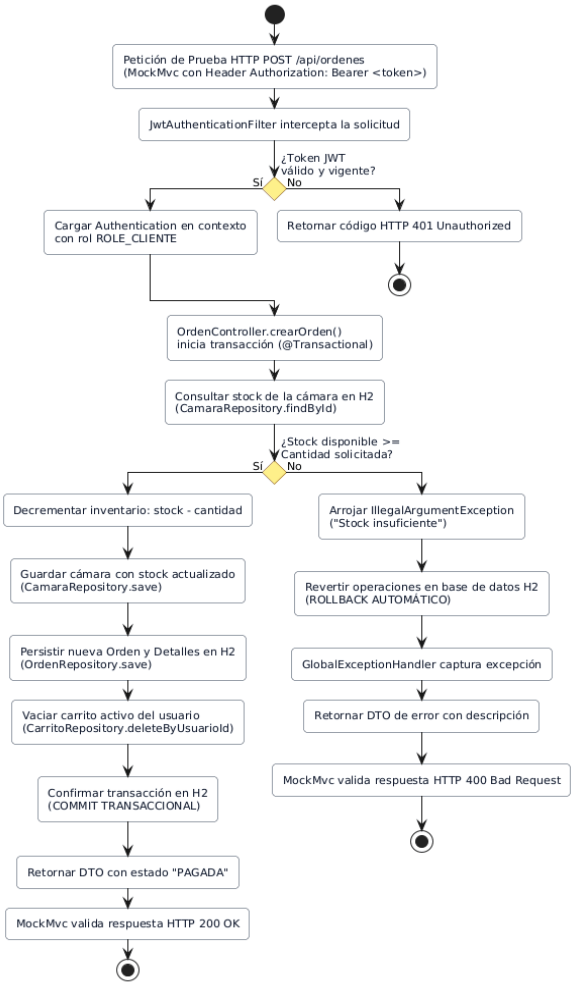
1.5.3 Diagrama del Modelo de Datos Relacional

El modelo relacional persistido en H2 para las pruebas de integración se compone de 7 entidades interrelacionadas con restricciones de integridad referencial:



1.5.4 Diagrama de Flujo de Servicios Integrados

El siguiente esquema detalla el recorrido exacto de una petición cliente-servidor para pruebas de integración automatizadas (MockMvc + Spring Data JPA + H2):


Petición de Prueba HTTP POST /api/ordenes
(MockMvc con Header Authorization: Bearer <token>)
JwtAuthenticationFilter intercepta la solicitud
¿Token JWT válido y vigente?
Sí
No
Cargar Authentication en contexto con rol ROLE_CLIENTE
Retornar código HTTP 401 Unauthorized
OrdenController.crearOrden() inicia transacción (@Transactional)
Consultar stock de la cámara en H2 (CamaraRepository.findById)
¿Stock disponible >= Cantidad solicitada?
Sí
No
Decrementar inventario: stock - cantidad
Guardar cámara con stock actualizado (CamaraRepository.save)
Persistir nueva Orden y Detalles en H2 (OrdenRepository.save)
Vaciar carrito activo del usuario (CarritoRepository.deleteByUsuarioId)
Confirmar transacción en H2 (COMMIT TRANSACCIONAL)
Retornar DTO con estado "PAGADA"
MockMvc valida respuesta HTTP 200 OK
Arrojar IllegalArgumentException ("Stock insuficiente")
Revertir operaciones en base de datos H2 (ROLLBACK AUTOMÁTICO)
GlobalExceptionHandler captura excepción
Retornar DTO de error con descripción
MockMvc valida respuesta HTTP 400 Bad Request

1.6 Objetivos del Proyecto de Aseguramiento de la Calidad

Objetivo General: Planificar, diseñar y validar el proceso integral de pruebas de software para la plataforma CamShop, asegurando la consistencia transaccional, la validación estricta de entradas y la estabilidad de las reglas de negocio antes de su despliegue comercial.

Objetivos Específicos:

- **1.** Definir el marco general de pruebas (planificación) estableciendo alcance delimitado (In-Scope / Out-of-Scope), tipos, entornos, cronograma general, matriz de riesgos operacionales y criterios de control.
- **2.** Diseñar el catálogo formal de casos de prueba unitarios para los servicios críticos de Spring Boot (JUnit 5 + Mockito) y componentes Angular (Jasmine + Karma).
- **3.** Diseñar casos de prueba de integración que verifiquen el flujo Controller-Service-Repository-H2 y el consumo de servicios cliente-servidor con interceptores.
- **4.** Estructurar la estrategia de automatización e Integración Continua (CI/CD) mediante pipelines en GitHub Actions para asegurar cobertura constante.

CAPÍTULO 2: FASE 1 — PLAN GENERAL DE PRUEBAS (PLANIFICACIÓN)

2.1 Documento de Alcance de Pruebas: In-Scope y Out-of-Scope

El alcance de pruebas cubre rigurosamente las funcionalidades críticas del flujo comercial de CamShop, delimitando claramente qué módulos se encuentran dentro del esfuerzo de evaluación (In-Scope) y cuáles quedan formalmente excluidos (Out-of-Scope) con su respectiva justificación técnica:

Módulo / Componente	Alcance	Justificación Técnica y Fronteras Operativas
M01: Autenticación y Cuentas (JWT)	**IN-SCOPE**	Pruebas sobre registro, hash BCrypt, emisión de tokens JWT, expiración y filtros de seguridad en endpoints.
M02: Catálogo de Cámaras	**IN-SCOPE**	Pruebas de filtrado reactivo por categoría, ordenamiento por precio y consulta de fichas técnicas en frontend y backend.
M03: Carrito de Compras	**IN-SCOPE**	Pruebas unitarias de cálculo de subtotales, totales reactivos con Signals y validación de tope por stock físico.
M04: Generación de Órdenes	**IN-SCOPE**	Pruebas de integración transaccional (@Transactional), descuento atómico de existencias y persistencia en H2.
M05: Pasarela de Pago Simulado	**IN-SCOPE**	Pruebas del algoritmo de cobro simulado, validación de tarjeta de prueba y actualización de estado a PAGADO.
M06: Reseñas y Calificaciones	**OUT-OF-SCOPE**	Para la fase de pruebas funcionales automatizadas, rendimiento y seguridad, este módulo queda fuera del alcance para concentrar los recursos de QA exclusivamente en el flujo crítico de negocio (End-to-End).
Pasarela Bancaria Real (Tarjetas)	**OUT-OF-SCOPE**	Ámbito académico demostrativo; no se efectúan transacciones bancarias reales con pasarelas de producción (Visa/Mastercard).
Streaming RTSP Cámaras Físicas	**OUT-OF-SCOPE**	El hardware físico de videocámaras está desacoplado del alcance web; se emplea un simulador interactivo en Angular.

2.2 Tipos de Prueba a Aplicar y Justificación Técnica

Siguiendo las recomendaciones del modelo de pirámide de pruebas de software (Mike Cohn), la inversión de esfuerzo se concentra en la base para asegurar máxima cobertura con ciclos rápidos de retroalimentación:

- • **Pruebas Unitarias (Caja Blanca / TDD):** Verifican métodos atómicos, algoritmos matemáticos (cálculo de carrito), hashing de credenciales y lógica reactiva de componentes sin dependencias de infraestructura ni accesos de red lentos.
- • **Pruebas de Integración (Bottom-Up):** Validan la colaboración fluida entre componentes adyacentes (Controladores REST consumiendo Servicios, Servicios interactuando con Repositorios JPA y H2 Database, e Interceptores HTTP en Angular).
- • **Integración Continua (CI/CD):** Automatización de toda la suite de pruebas en cada push o pull request mediante un pipeline en la nube de GitHub Actions, garantizando compilación limpia y detección inmediata de regresiones.

2.3 Entorno de Pruebas: Hardware, Software, Datos y Herramientas

Cuadro oficial de herramientas y tecnologías homologadas para el aseguramiento de la calidad de CamShop:

Categoría / Nivel	Herramienta / Framework	Versión / Configuración de Entorno
Pruebas Unitarias Backend	JUnit 5 (Jupiter) + Mockito	Versión 5.10 / Java 17 LTS / OpenJDK
Pruebas Integración Backend	Spring Boot Test + MockMvc	Spring Boot 3.x / H2 Database en memoria
Pruebas Unitarias Frontend	Jasmine + Karma Runner	Jasmine 5.9 / Node.js v20+ / Angular CLI 20
Navegador de Ejecución Headless	ChromeHeadless / Edge CLI	Modo headless (--no-sandbox, --disable-gpu)
Pipeline de Integración Continua	GitHub Actions Workflows	Ubuntu-latest runner / Maven + Node actions
Control de Versiones y Trazabilidad	Git + Repositorio GitHub	Branching model con pull request obligatorios

2.4 Cronograma General del Proyecto

El siguiente cronograma integral articula las 18 semanas académicas del curso, sincronizando con exactitud los hitos de desarrollo de software, las actividades específicas de aseguramiento de calidad (QA) y los despliegues automatizados correspondientes:

Sem.	Hito / Entregable de Desarrollo	Fase y Actividades de Prueba (QA)	Despliegue / CI/CD	Responsables
1 - 2	Levantamiento de Requisitos (SRS) y Arquitectura Base del Sistema.	**Definición del Plan de Pruebas, Alcance (In/Out-Scope) y Matriz de Riesgos Inicial.**	Setup de repositorio Git y branch main.	Todos
3 - 4	Desarrollo Backend: Modelos JPA, AuthService y CamaraService.	**Diseño y Ejecución de Pruebas Unitarias Backend (JUnit 5 + Mockito sobre auth y catálogos).**	Configuración inicial de maven test en CI.	Arnold Lujan
5 - 6	Desarrollo Frontend: Componentes Hero, Catálogo, Login y Carrito.	**Diseño de Pruebas Unitarias Frontend (Jasmine/Karma) e Integración con MockMvc y H2.**	Pipeline CI/CD en GitHub Actions activo.	Martin Gonzales
7 - 8	Integración End-to-End: Carrito, Checkout y Generación de Órdenes.	**Pruebas de Transaccionalidad (@Transactional), Integración HTTP y Pruebas Funcionales.**	Despliegue v1.0-alpha en entorno de staging.	Ambos
9 - 10	Desarrollo del flujo End-to-End crítico (Login hasta Checkout).	**Pruebas Funcionales Automatizadas (Selenium/Cypress) y Pruebas de Rendimiento (JMeter) enfocadas en el flujo E2E de compra.**	Despliegue v1.0-alpha en entorno de staging.	Arnold Lujan
11 - 12	Refactorización y robustecimiento de seguridad en APIs y cabeceras.	**Pruebas de Seguridad en Formularios y APIs REST (OWASP Top 10, ZAP, inyección SQL).**	Escaneo estático de seguridad (SAST).	Martin Gonzales
13 - 14	Optimización de Responsive Design y Accesibilidad Web.	**Pruebas de Usabilidad, Accesibilidad (Lighthouse, WAVE) y Cross-Browser.**	Despliegue v1.0-beta para pruebas de usuario.	Ambos
15 - 16	Congelamiento de código (Code Freeze) y corrección de defectos menores.	**Pruebas de Regresión Completa Automatizada sobre todas las suites unitarias y de integración.**	Despliegue de Release Candidate (v1.0-RC).	Ambos
17 - 18	Preparación de entregables finales y material de presentación.	**Consolidación del Informe Final de Calidad, Métricas de Cobertura y Sustentación.**	Release final v1.0 en GitHub.	Todos

2.5 Matriz de Gestión de Riesgos del Producto y Riesgo Operacional por Módulo

La gestión de riesgos se aborda desde dos perspectivas complementarias: los riesgos técnicos generales del proyecto y la Matriz de Riesgo Operacional por Módulo (Risk-Based Testing), que orienta el nivel de cobertura requerido:

2.5.1 Matriz de Riesgos Técnicos del Proyecto

Se evaluaron los riesgos técnicos asignando Probabilidad (P: 1 a 5) e Impacto (I: 1 a 5), obteniendo el Nivel de Riesgo (R = P x I):

ID	Descripción del Riesgo Técnico	P	I	R	Plan de Mitigación y Acción Preventiva
RSK-01	Inyección de tokens JWT expirados o con firma adulterada vulnerando endpoints.	2	5	**10 (Alto)**	Pruebas unitarias sobre JwtService validando algoritmo HMAC-SHA256 y expiración.
RSK-02	Inconsistencia en stock de inventario por compras concurrentes simultáneas.	3	4	**12 (Alto)**	Pruebas de integración sobre OrdenService con control transaccional y rollback.
RSK-03	Errores aritméticos de redondeo en el cálculo de subtotales e impuestos del carrito.	2	4	**8 (Medio)**	Técnica de análisis de valores límite y pruebas unitarias con precisión decimal.
RSK-04	Fallo de ejecución de pruebas en servidores CI de GitHub Actions por falta de display gráfico.	3	3	**9 (Medio)**	Estandarización de ChromeHeadless con flags --no-sandbox y xvfb en workflows.

2.5.2 Matriz de Riesgos Operacionales y Cobertura de Pruebas por Módulo

Con el objetivo de maximizar el retorno del esfuerzo de QA, se clasifica cada módulo según su criticidad operacional para el negocio, asignando una meta de cobertura de pruebas obligatoria:

Módulo de Software	Impacto de Falla Operacional	Nivel de Riesgo	Cobertura Requerida	Estrategia de Prueba Priorizada
M01: Autenticación y Cuentas	Acceso no autorizado, suplantación de identidad y fuga de datos personales de clientes.	**CRÍTICO**	**>= 90%**	Unitarias + Integración Security + Pruebas de Seguridad (OWASP ZAP)
M04: Órdenes y Checkout	Pérdida económica directa, cobros no registrados o inventario descontado erróneamente.	**CRÍTICO**	**>= 90%**	Integración Transaccional (@Transactional) + Pruebas Funcionales E2E y Rendimiento (JMeter)

M03: Carrito de Compras	Venta por encima del stock disponible o montos cobrados incorrectos al comprador.	**ALTO**	**>= 85%**	Unitarias Límite + Signals
M02: Catálogo de Cámaras	Inconsistencia en precios mostrados o fallos de visualización que impidan la compra.	**MEDIO**	**>= 75%**	Unitarias + HttpClientTesting
M06: Reseñas y Calificaciones	Distorsión del promedio de estrellas o spam en comentarios de productos.	**BAJO**	**>= 70%**	No aplica para APF2 (Flujo Secundario)

2.6 Criterios de Entrada, Salida y Suspensión / Reanudación

- • **Entrada:** Criterio de Entrada: Repositorio en GitHub sincronizado en rama main/release, código fuente compila con cero advertencias críticas y base de datos H2 aprovisionada en memoria.
- • **Salida:** Criterio de Salida: 100% de casos críticos ejecutados con tasa de aprobación >= 95%, cero defectos de severidad Bloqueante/Alta abiertos y cobertura mínima general de código >= 80%.
- • **Suspensión:** Criterio de Suspensión: Se suspenderá la fase de pruebas si más del 25% de los casos fallan consecutivamente por inestabilidad de infraestructura, fallas de dependencias o caída de la base de datos.
- • **Reanudación:** Criterio de Reanudación: Se reanudará tras la resolución técnica formal del bloqueo y la verificación exitosa mediante una prueba de humo automatizada (Smoke Test).

CAPÍTULO 3: FASE 2 — DISEÑO DE CASOS DE PRUEBA (UNIDAD 1)

3.1 Matriz de Trazabilidad y Técnicas de Diseño Aplicadas

Para el diseño formal de casos de prueba se emplearon técnicas de Caja Negra (Partición de Equivalencia y Análisis de Valores Límite) combinadas con técnicas de Caja Blanca (cobertura de ramas, aislamiento con Mocks y verificación de estados reactivos):

Código RF	Descripción del Requisito Funcional	Técnica de Diseño Aplicada
RF-01	Registro de clientes con correo electrónico único y contraseña >= 6 caracteres.	Partición de Equivalencia (Clases Válidas e Inválidas)
RF-02	Autenticación, generación de JWT con claims y roles válidos.	Caja Blanca con Aislamiento de Mocks (Mockito)
RF-03	Filtrado dinámico del catálogo por precio, categoría y stock.	Partición de Equivalencia sobre rangos numéricos
RF-04	Adición al carrito, validación de stock y cálculo de subtotales.	Análisis de Valores Límite (Límite superior)
RF-05	Creación de orden transaccional con pago simulado y descuento de stock.	Integración Transaccional con Verificación en BD H2
RF-06	Publicación de reseñas y cálculo de promedio de reputación.	Valores Límite (Extremos 1 y 5 estrellas)

3.2 Catálogo de Casos de Prueba Unitarios Backend

A continuación, se detallan las fichas técnicas formales de los casos unitarios de backend:

ID del Caso:	**CP-UNI-001**
Módulo / Tipo:	Autenticación (AuthService) / Prueba Unitaria Backend
Requisito Asociado:	RF-01: Registro de nuevo usuario con formato de correo y clave válidos.
Técnica de Diseño:	Partición de Equivalencia (Clase válida) + Mocks (Mockito).
Precondición:	Servicio AuthService instanciado; UserRepository simulado sin usuario previo.
Datos de Entrada:	RegisterRequest: { username: "cliente.qa@camshop.pe", password: "Password123!" }
Pasos de Ejecución:	1) Configurar mock existsByUsername = false. 2) Invocar authService.register(request). 3) Verificar llamada a save().
Resultado Esperado:	El método ejecuta exitosamente, encripta la contraseña con BCrypt, asigna ROLE_CLIENTE y persiste el registro.
Prioridad:	**Alta**

ID del Caso:	**CP-UNI-002**
Módulo / Tipo:	Autenticación (AuthService) / Prueba Unitaria Backend

Requisito Asociado:	RF-01: Rechazo de registro duplicado de cuenta.
Técnica de Diseño:	Partición de Equivalencia (Clase inválida) + Aserción de Excepción.
Precondición:	El correo ingresado ya existe previamente registrado en base de datos.
Datos de Entrada:	RegisterRequest: { username: "cliente.existente@camshop.pe", password: "Password123!" }
Pasos de Ejecución:	1) Configurar mock existsByUsername = true. 2) Ejecutar authService.register(request).
Resultado Esperado:	El servicio intercepta el duplicado y lanza ConflictException con mensaje descriptivo. No se invoca save().
Prioridad:	**Alta**

ID del Caso:	**CP-UNI-003**
Módulo / Tipo:	Carrito de Compras (CarritoService) / Prueba Unitaria Backend
Requisito Asociado:	RF-04: Cálculo aritmético de subtotal e incremento de cantidad.
Técnica de Diseño:	Análisis de Valores Límite + Caja Blanca.
Precondición:	Existe un cliente ID 1 y una cámara ID 10 con Precio S/. 200.00 y Stock 10.
Datos de Entrada:	camaraId: 10, cantidad: 3.
Pasos de Ejecución:	1) Simular cámara con precio S/. 200.00. 2) Invocar carritoService.agregarItem(1, 10, 3). 3) Verificar subtotal.
Resultado Esperado:	El subtotal del ítem es exactamente 3 x 200.00 = 600.00 y el monto total acumulado del carrito se actualiza.
Prioridad:	**Alta**

ID del Caso:	**CP-UNI-004**
Módulo / Tipo:	Carrito de Compras (CarritoService) / Prueba Unitaria Backend
Requisito Asociado:	RF-04: Restricción de compra al superar el stock físico disponible.
Técnica de Diseño:	Análisis de Valores Límite (Límite superior excedido: Stock = 5, Solicitado = 6).
Precondición:	La cámara solicitada cuenta únicamente con 5 unidades físicas en inventario.
Datos de Entrada:	camaraId: 10, stockDisponible: 5, cantidadSolicitada: 6.
Pasos de Ejecución:	1) Configurar cámara con stock = 5. 2) Invocar carritoService.agregarItem(1, 10, 6).
Resultado Esperado:	El servicio deniega la operación arrojando IllegalArgumentException por falta de existencias.
Prioridad:	**Alta**

ID del Caso:	**CP-UNI-005**
Módulo / Tipo:	Catálogo de Cámaras (CamaraService) / Prueba Unitaria Backend

Requisito Asociado:	RF-03: Filtrado de cámaras por rango de precios.
Técnica de Diseño:	Partición de Equivalencia sobre rangos numéricos.
Precondición:	Existen cámaras registradas con precios S/. 100.00, S/. 250.00 y S/. 600.00.
Datos de Entrada:	precioMin: 150.00, precioMax: 400.00.
Pasos de Ejecución:	1) Invocar camaraService.listarConFiltros(null, 150.00, 400.00). 2) Evaluar lista resultante.
Resultado Esperado:	Retorna únicamente los productos que cumplen 150 <= precio <= 400 (la cámara de S/. 250.00).
Prioridad:	**Media**

3.3 Catálogo de Casos de Prueba Unitarios Frontend (Jasmine + Karma)

ID del Caso:	CP-UNI-006
Módulo / Tipo:	Landing Page (HeroComponent) / Prueba Unitaria Frontend
Requisito Asociado:	RF-03: Conmutación interactiva de canales en simulador CCTV.
Técnica de Diseño:	Pruebas de Transición de Estados en Angular Signals.
Precondición:	Componente HeroComponent montado en el DOM virtual de pruebas.
Datos de Entrada:	Selección del canal CAM-02 (Perímetro Exterior).
Pasos de Ejecución:	1) Verificar selectedCamera().id === "CAM-01". 2) Invocar selectCamera(cameras[1]). 3) Ejecutar detectChanges().
Resultado Esperado:	La señal selectedCamera se actualiza reactivamente a CAM-02 con resolución 4K Ultra y modo nocturno.
Prioridad:	Media

ID del Caso:	CP-UNI-007
Módulo / Tipo:	Landing Page (HeroComponent) / Prueba Unitaria Frontend
Requisito Asociado:	RF-03: Actualización en tiempo real del reloj de monitoreo CCTV.
Técnica de Diseño:	Verificación de Temporizadores Virtuales y Limpieza en ngOnDestroy.
Precondición:	Componente montado con setInterval activo cada 1000ms.
Datos de Entrada:	Avance virtual de tiempo: jasmine.clock().tick(1000).
Pasos de Ejecución:	1) Instalar reloj virtual. 2) Inicializar componente. 3) Avanzar 1s. 4) Validar señal currentTimestamp().
Resultado Esperado:	La señal contiene una cadena válida en formato HH:mm:ss y al invocar ngOnDestroy() el temporizador se cancela.
Prioridad:	Baja

ID del Caso:	CP-UNI-008
Módulo / Tipo:	Landing Page (SolucionesComponent) / Prueba Unitaria Frontend
Requisito Asociado:	RF-03: Filtrado dinámico de soluciones según entorno de uso.
Técnica de Diseño:	Partición de Equivalencia sobre colecciones reactivas.
Precondición:	El componente tiene 3 soluciones base (Interior, Exterior, PTZ).
Datos de Entrada:	filtro: "interior".
Pasos de Ejecución:	1) Ejecutar component.filtrar("interior"). 2) Evaluar getter computado solucionesFiltradas.
Resultado Esperado:	El arreglo computado contiene exactamente 1 elemento con ID "interior". Al filtrar "todos" devuelve los 3.

Prioridad:	**Media**

ID del Caso:	**CP-UNI-009**
Módulo / Tipo:	Autenticación (LoginComponent) / Prueba Unitaria Frontend
Requisito Asociado:	RF-02: Alternador de visibilidad de contraseña con Angular Signal.
Técnica de Diseño:	Pruebas de Estado Reactivo y Modificación de Atributos en DOM.
Precondición:	LoginComponent montado con showPassword() inicializado en false.
Datos de Entrada:	Evento clic en botón con ícono de ojo.
Pasos de Ejecución:	1) Verificar input con type="password". 2) Invocar togglePassword(). 3) Ejecutar detectChanges().
Resultado Esperado:	La señal showPassword cambia a true y el atributo en el DOM pasa dinámicamente a type="text".
Prioridad:	**Media**

3.4 Catálogo de Casos de Prueba de Integración Backend

ID del Caso:	CP-INT-010
Módulo / Tipo:	API REST (/api/auth/register) / Prueba de Integración Backend
Requisito Asociado:	RF-01: Registro integral cliente-servidor con persistencia real en H2.
Técnica de Diseño:	Integración Bottom-Up (Controller + Service + Repository + H2 Database).
Precondición:	Base de datos H2 en memoria levantada y limpia de usuarios de prueba.
Datos de Entrada:	POST /api/auth/register con payload: {"username": "qa.integracion@camshop.pe", "password": "Password123!"}
Pasos de Ejecución:	1) Enviar POST vía mockMvc.perform(). 2) Validar status HTTP. 3) Consultar tabla con userRepository.
Resultado Esperado:	Código HTTP 200 OK. El usuario queda efectivamente persistido en H2 con contraseña encriptada con BCrypt.
Prioridad:	Alta

ID del Caso:	CP-INT-011
Módulo / Tipo:	API REST (/api/auth/login) / Prueba de Integración Backend
Requisito Asociado:	RF-02: Emisión y validación de Token JWT tras autenticación correcta.
Técnica de Diseño:	Integración de Seguridad, Criptografía y Filtros Spring Security.
Precondición:	Usuario previamente persistido en H2 con credenciales válidas.
Datos de Entrada:	POST /api/auth/login con credenciales correctas de cliente.
Pasos de Ejecución:	1) Enviar POST con mockMvc. 2) Extraer JSON de respuesta. 3) Validar estructura del token.
Resultado Esperado:	Código HTTP 200 OK. El cuerpo contiene un campo token con estructura JWT válida (3 segmentos separados por puntos).
Prioridad:	Alta

ID del Caso:	CP-INT-012
Módulo / Tipo:	API REST (/api/ordenes) / Prueba de Integración Backend
Requisito Asociado:	RF-05: Transaccionalidad de orden, pago simulado y descuento de inventario.
Técnica de Diseño:	Integración Transaccional con Verificación de Efectos Secundarios en BD.
Precondición:	Cliente autenticado con carrito activo de 2 unidades de CAM-01 (Stock inicial: 10).
Datos de Entrada:	POST /api/ordenes con Header Authorization: Bearer <token_valido>.
Pasos de Ejecución:	1) Ejecutar creación de orden. 2) Consultar tabla órdenes. 3) Consultar stock de CAM-01 en H2.
Resultado Esperado:	Código HTTP 200 OK. Se registra la orden como PAGADA, el carrito queda vacío y el stock pasa a 8 unidades.

Prioridad:	**Alta**

3.5 Catálogo de Casos de Prueba de Integración Frontend

ID del Caso:	**CP-INT-013**
Módulo / Tipo:	Servicio Catálogo (CamaraService) / Prueba de Integración Frontend
Requisito Asociado:	RF-03: Consumo de endpoints REST y deserialización de modelos tipados.
Técnica de Diseño:	Simulación de Capa de Transporte HTTP con HttpTestingController.
Precondición:	CamaraService configurado con provideHttpClientTesting().
Datos de Entrada:	Invocación a camaraService.listarCamaras().
Pasos de Ejecución:	1) Suscribirse al método. 2) Interceptar petición GET a /api/camaras. 3) Emitir 2 cámaras mock. 4) Verificar datos.
Resultado Esperado:	La petición saliente es GET, la URL coincide y el servicio deserializa exitosamente el arreglo en Camara[].
Prioridad:	**Alta**

ID del Caso:	**CP-INT-014**
Módulo / Tipo:	Seguridad de Red (JwtInterceptor) / Prueba de Integración Frontend
Requisito Asociado:	RF-02: Inyección obligatoria de cabecera Authorization en rutas protegidas.
Técnica de Diseño:	Verificación de Interceptores HTTP en Angular.
Precondición:	Token JWT almacenado en el almacenamiento de sesión local (localStorage).
Datos de Entrada:	Petición HTTP a endpoint protegido /api/cliente/perfil.
Pasos de Ejecución:	1) Ejecutar petición con interceptor activo. 2) Capturar solicitud saliente con httpMock.expectOne().
Resultado Esperado:	La solicitud enviada al servidor contiene obligatoriamente la cabecera Authorization: Bearer <token>.
Prioridad:	**Alta**

CAPÍTULO 4: FASE 2 — DISEÑO DE CASOS DE PRUEBA (UNIDAD 2)

4.1 Catálogo de Casos de Prueba Funcionales E2E

Como base para la automatización funcional, pruebas de rendimiento y seguridad del siguiente avance, se define la ficha técnica formal del flujo crítico integral de compra:

ID del Caso:	**CP-FUNC-E2E-01**
Módulo / Tipo:	Flujo Crítico de Negocio / Prueba Funcional Automatizada (E2E)
Requisito Asociado:	RF-02 (Login), RF-03 (Catálogo), RF-04 (Carrito), RF-05 (Checkout).
Técnica de Diseño:	Pruebas basadas en Casos de Uso (Flujo Básico / Happy Path)
Precondición:	El sistema debe estar corriendo y debe existir un usuario de prueba registrado y una cámara con stock en la base de datos.
Datos de Entrada:	Usuario: cliente.qa@camshop.pe | Clave: Password123! | Producto: CAM-01
Pasos de Ejecución:	1) Abrir navegador y navegar a la página de Login. 2) Ingresar correo y contraseña, presionar "Ingresar". 3) Navegar a la página del "Catálogo" y seleccionar cámara CAM-01. 4) Clic en "Agregar al Carrito" (1 unidad). 5) Navegar a la vista del "Carrito de Compras". 6) Clic en "Proceder al Pago / Checkout". 7) Confirmar orden de compra simulada.
Resultado Esperado:	El sistema muestra mensaje de "Compra exitosa", el estado de la orden es "PAGADA", y el carrito queda vacío (0 ítems).
Prioridad:	**Alta**

REFERENCIAS BIBLIOGRÁFICAS

1. International Software Testing Qualifications Board (ISTQB). (2018). Certified Tester Foundation Level Syllabus. ISTQB.

2. Pressman, R. S. (2010). Ingeniería de Software: Un enfoque práctico (7.ª ed.). McGraw-Hill Interamericana.

3. Sommerville, I. (2011). Ingeniería del Software (9.ª ed.). Pearson Educación.

4. Piattini Velthuis, M. G. (2014). Calidad de Sistemas de Información (4.ª ed.). Ediciones de la U.

5. OWASP Foundation. (2021). OWASP Top 10: The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/

6. IEEE Computer Society. (2008). IEEE Standard for Software and System Test Documentation (IEEE Std 829-2008). IEEE.